

Linear Algebra

Broadcasting

Add vector $\mathbf{b}$ to every column of matrix A :

$$C = A + \mathbf{b}, \quad C_{\{i,j\}} = A_{\{i,j\}} + b_j$$

Matrix multiplication

For vectors:

$$\mathbf{x}^T \mathbf{y} = \|\mathbf{x}\|_2 \|\mathbf{y}\|_2 \cos(\theta)$$

Hadamard product: Element-wise multiplication: $\mathbf{A} \odot \mathbf{B}$

Norms

- **L^p norm:** $\|\mathbf{x}\|_p = \left(\sum_i |x_i|^p\right)^{\frac{1}{p}}$
- **L^∞ norm:** maximum element of vector, $\|\mathbf{x}\|_\infty = \max_i |x_i|$
- **Frobenius norm:** L^2 norm for matrices, $\|\mathbf{A}\|_F = \sqrt{\sum_{i,j} A_{i,j}^2}$

Diagonal Matrices

A Diagonal matrix is all zero except for its diagonal. The diagonal we call vector $\mathbf{v}$:

$$\mathbf{v} = \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}$$
$$\text{diag}(\mathbf{v}) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Multiplying a diagonal matrix with a vector is computationally efficient:

$$\text{diag}(\mathbf{v})\mathbf{x} = \mathbf{v} \odot \mathbf{x}$$

Inverting diagonal matrices

$$\text{diag}(\mathbf{v})^{-1} = \text{diag}\left(\left(\frac{1}{v_1} \quad \dots \quad \frac{1}{v_n}\right)^T\right)$$

Orthogonal matrices

$$\mathbf{A}^T \mathbf{A} = \mathbf{A} \mathbf{A}^T = \mathbf{I}$$
$$\mathbf{A}^{-1} = \mathbf{A}^T$$

Eigenvectors

A real symmetric matrix with $\mathbf{n}$ rows and columns has $\mathbf{n}$ real eigenvalues and $\mathbf{n}$ orthogonal eigenvectors.

Eigenvectors can be concatenated into a $\mathbf{n}$ orthogonal matrix:

$$\mathbf{V} = (\mathbf{v}^{(1)} \quad \dots \quad \mathbf{v}^{(n)})$$

The eigenvalues can be concatenated into a vector:

$$\boldsymbol{\lambda} = (\lambda_1 \quad \dots \quad \lambda_n)^T$$

Then the matrix A can be rewritten as:

$$A = V \operatorname{diag}(\lambda) V^{-1}$$

Optimization using Eigendecomposition

The optimization problem of maximizing or minimizing $f(x) = x^T A x$ subject to $\|x\|_2 = 1$ can be solved using eigendecomposition:

$$\begin{aligned} Ax &= \lambda x \\ x^T A x &= x^T \lambda x = \lambda x^T x = \lambda \end{aligned}$$

- **Maximum:** eigenvector belonging to the **maximum eigenvalue**.
- **Minimum:** eigenvector belonging to the **minimum eigenvalue**.

Probability Theory

Conditional Theory

$$P(y = y \mid x = x) = \frac{P(y = y, x = x)}{P(x = x)}$$

Three chained together:

$$\begin{aligned} P(a, b, c) &= P(a \mid b, c)P(b, c) \\ P(b, c) &= P(b \mid c)P(c) \\ P(a, b, c) &= P(a \mid b, c)P(b \mid c)P(c) \end{aligned}$$

Bayes Theory

$$P(x \mid y) = \frac{P(x)P(y \mid x)}{P(y)}$$

Rule of total probability

$$P(y) = \sum_x P(y \mid x)P(x)$$

Expectation

$$\begin{aligned} E[x] &= \sum_x xP(x) \\ E[f(x)] &= \sum_x f(x)P(x) \end{aligned}$$

Linearity of Expectation:

$$E[ax + b] = aE[x] + b$$

Variance

The **variance** σ^2 measures the variability around the expectation. σ is the **standard deviation** and is the square root of the variance.

$$\operatorname{Var}(f(x)) = \mathbb{E}[(f(x) - \mathbb{E}[f(x)])^2]$$

Covariance

The covariance σ_{xy} is the linear relation between two random variables x and y .

Numerical Computation

General Methodology in Deep Learning

- m training data points $x^{(1)}, \dots, x^{(m)}$
- Empirical data distribution: $\hat{p}(x) = \frac{1}{m} \sum_{i=1}^m \delta(x - x^{(i)})$
- Neural Network predicts distribution: $p_{\text{model}(x;\theta)}$
 - Where θ are the learnable parameters
- Training: Make $p_{\text{model}(x)}$ as similar as possible to $\hat{p}(x)$

→ Minimize **KL-divergence** (or equivalently the **cross entropy**).

→ Learning is an optimization problem

Quantization

A PC only has a **finite number of bits** available, therefore we get rounding errors. The accumulation of said rounding errors can be problematic.

Underflow: The rounding of small numbers to zero. Not a direct problem, but further computation can be problematic, like $\frac{1}{x}$ or $\log(x)$ as $x \rightarrow 0$

Overflow: The rounding of large numbers to ∞ . No further computation possible.

Stabilizing Softmax

For example, softmax needs to be stabilized, as very high x result in $\exp(x)$ infinity or very negative x result in $\exp(x)$ 0:

$$\text{softmax}(x)_i = \frac{\exp(x_i)}{\sum_{j=1}^n \exp(x_j)}$$

If we **add a constant** value **k** to all inputs, we do not change the output:

$$\frac{\exp(x_1 + k)}{\exp(x_1 + k) + \exp(x_2 + k)} = \frac{\exp(x_1) \exp(k)}{\exp(x_1) \exp(k) + \exp(x_2) \exp(k)}$$

Softmax is stabilized by subtracting the maximum input from all elements:

$$z = x - \max_i x_i$$

Poor Conditioning

Small change of the input has a large effect on the output.

Gradient-Based Optimization

In ML we try to minimize the cross-entropy H between the empirical distribution and the parametric model distribution. Minimizing the cross-entropy is equivalent to minimizing the KL-divergence.

$$\min_{\theta} H(\hat{p}_{\text{data}}, p_{\text{model}(\theta)})$$

Or anders:

$$f(x) = H(\hat{p}_{\text{data}}, p_{\text{model}(x)})$$
$$x^* = \arg \min f(x)$$

Algorithm of Gradient Descent

1. Initialize to some point x_0
2. Calculate derivative $f'(x_0)$
3. Take small step ε in the negative direction of the derivative
 - $x_1 = x_0 - \varepsilon f'(x_0)$
 - $f(x_1) \approx f(x_0) - \varepsilon f'(x_0)$
4. Iterate until $f'(x_0) = 0$

In multidimensional space:

$$\mathbf{x}' = \mathbf{x} - \varepsilon \nabla_{\mathbf{x}} f(\mathbf{x})$$

Machine Learning Basics

Linear Regression

A linear regression can be expressed with:

$$\hat{y} = \mathbf{w}^T \mathbf{x}$$

where $\mathbf{w}$, $\mathbf{x}$ have the dimension of feature length. If there is only one feature to evaluate, the result is $\hat{y} = ax$, which fits the mental model of conventional linear regression. If we evaluate all data points $\mathbf{X}$ with all features, we get:

$$\hat{\mathbf{y}} = \mathbf{X}\mathbf{w}$$

Performance measure (loss)

We need some kind of loss function, for example **MSE**:

$$\begin{aligned} \text{MSE}_{\text{test}} &= \frac{1}{m} \sum_i (\hat{\mathbf{y}}^{(\text{test})} - \mathbf{y}^{(\text{test})})_i^2 \\ &= \frac{1}{m} \|\hat{\mathbf{y}}^{(\text{test})} - \mathbf{y}^{(\text{test})}\|_2^2 \end{aligned}$$

Capacity, Overfitting and Underfitting

The test error rate will typically never be 0, even if we knew the underlying distribution p_{data} (measurement errors, missing relation between x and y , ...). The resulting test error of a perfect classifier is the **Bayes error rate/irreducible error**.

The No Free Lunch Theorem

“No machine learning algorithm is universally any better than any other.”